

SIMATS ENGINEERING
ASSESSMENT TOOL 2: Case Study

COURSE NAME: COMPUTER VISION

COURSE CODE:ITA0515

Slot B

COURSE OUTCOME COVERED

CO3: Analyze and extract image features using detection and matching techniques for effective image representation and comparison. (BTL 4)

Assessment Tool Weightage: 40%

QUESTIONS:5

Total Marks 50

Duration :60 Minutes

Annexure A

Case Study

Code of Conduct:

I __A.SHIVA SHANKAR REDDY(192425174) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.I understand that any violation of academic integrity rules will result in disciplinary action.

A.Shiva Shankar Reddy

Signature of the student:

Rubrics for Evaluation

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Marks(100)
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly	
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts	
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning	
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided	

Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand	
---------	----	--	---	-------------------------------	---------------------------------------	--

1. Preprocessing and Feature Detection

Case: A vision system processes images captured under varying lighting conditions. Without preprocessing, the feature detection results are inconsistent. Analyze the issue and explain how preprocessing improves feature detection performance.

Answer:

The main issue in this case is that **varying lighting conditions change the intensity and appearance of image features**, causing the feature detector to identify different features in otherwise similar images. When an image becomes brighter, darker, or has uneven illumination, important edges, corners, and other local structures may become less distinct. This directly leads to inconsistent feature detection.

Preprocessing helps by making the input images more suitable and consistent before feature detection is performed. For example, **grayscale conversion** simplifies the image by representing intensity information, while **noise reduction** can remove unwanted variations that may be incorrectly detected as features. Contrast enhancement can also improve the visibility of important structures when the lighting is poor or uneven.

The purpose of these operations is not to create new features, but to reduce variations that interfere with the detection process. As a result, the same important structures are more likely to be detected consistently under different lighting conditions.

Therefore, preprocessing is an important step in this case because it improves the **stability, reliability, and accuracy of feature detection**. By reducing the effect of illumination variations before detection, the vision system can produce more consistent image features and consequently improve subsequent image analysis and comparison.

2. Edge vs Corner Detection

In an image containing both straight edges and textured regions, a system uses edge detection but fails to identify key points for matching. Analyze why edge detection is insufficient and justify the use of corner detection in this case.

Answer:

In the given case, edge detection is insufficient because its primary purpose is to identify **boundaries or regions with significant intensity changes**, rather than distinctive points that can be reliably used for feature matching. Although the system can detect the straight edges present in the image, points along a single straight edge may not be unique enough to establish reliable correspondence between images.

This limitation becomes important when the objective is to identify **key points for matching**. A corner provides intensity changes in more than one direction and therefore represents a more distinctive local structure. Such points are easier to identify and compare across images.

For the textured regions in the case, corner detection can identify locations where image intensity changes significantly in multiple directions. These locations provide more useful information than a continuous edge because they can serve as identifiable key points.

Aspect	Edge Detection	Corner Detection
Main purpose	Detect boundaries and edges	Detect distinctive key points
Information	Mainly identifies intensity change along an edge	Identifies intensity changes in multiple directions
Suitability for matching	Limited when points along an edge are not distinctive	More suitable for key-point matching
Application in this case	Detects straight edges but misses useful key points	Identifies distinctive points in textured regions

Therefore, the failure is not that edge detection is incorrect; rather, **it is not sufficient for the specific requirement of feature matching**. Corner detection is justified because the case requires distinctive key points rather than only boundaries. A combination of edge and corner detection may also be useful when both structural boundaries and matching points are required.

3. Hough Transform in Noisy Images

A system uses Hough Transform to detect lines in noisy images but produces inaccurate results. Analyze the effect of noise on detection and suggest improvements based on preprocessing or parameter selection.

The inaccurate line detection in this case is mainly caused by the presence of **noise in the input images**. The Hough Transform identifies geometric structures such as lines by analyzing image points and accumulating evidence for possible line parameters. When noise is present, unwanted pixels or false edges can also contribute to this process.

Consequently, noise may create additional peaks in the Hough parameter space. The system can therefore interpret these unwanted responses as actual lines, resulting in **false detections or inaccurate line locations**. The problem becomes more significant when the noisy image contains many irrelevant edges.

A suitable improvement is to perform preprocessing before applying the Hough Transform. **Noise filtering** can reduce unwanted variations, and an appropriate edge-detection stage can help provide cleaner edge information to the Hough Transform. This allows the transform to work mainly with meaningful image structures rather than noisy points.

Parameter selection is also important. The **accumulator threshold** should be selected appropriately so that weak responses caused by noise are not treated as valid lines. Depending on the implementation, parameters controlling the minimum line length or allowable gaps can also be adjusted to reject insignificant detections.

Therefore, the most suitable solution for the case is to combine **effective preprocessing with appropriate Hough Transform parameters**. Reducing noise before transformation and selecting parameters according to the image characteristics can significantly reduce false lines and improve the reliability of line detection.

4. Feature Matching under Transformations

Two images of the same object are taken at different scales and orientations. Traditional feature matching fails, but scale invariant feature matching works effectively. Analyze why scale invariant methods perform better in this scenario.

Answer:

Traditional feature matching can fail in this case because the same object does not have exactly the same visual appearance when it is captured at different **scales and orientations**. Changing the scale alters the size of the features, while changing the orientation changes their spatial arrangement and appearance. A matching method that depends strongly on the original feature size or orientation may therefore fail to establish correct correspondences.

Scale-invariant feature matching methods are specifically designed to handle such transformations. Instead of relying only on the direct appearance or position of a feature, these methods identify distinctive local features and represent them using descriptors that are more robust to changes in scale and orientation.

In the given case, the two images still contain the same underlying object information even though the object's representation has changed. A scale-invariant method can recognize corresponding local structures despite these changes. This explains why the scale-invariant approach succeeds while traditional matching fails.

The comparison can be summarized as follows:

Factor	Traditional Feature Matching	Scale-Invariant Matching
Scale changes	May cause matching failure	Designed to handle scale changes
Orientation changes	Can-reduce-correspondence accuracy	More robust to orientation changes
Feature correspondence	Sensitive to appearance changes	More reliable under transformations
Suitability for this case	Less suitable	More suitable

Therefore, scale-invariant feature matching performs better because the case specifically involves **changes in scale and orientation**. Its robustness to these transformations allows corresponding features of the same object to be identified more reliably

5. PCA for Feature Optimization

A system extracts a large number of features, leading to high computational cost and redundancy. Analyze how Principal Component Analysis helps in reducing dimensionality and improving efficiency.

The problem in this case is that extracting a large number of features increases the amount of data that must be stored and processed. Some of these features may also contain **redundant or correlated information**. Processing all of them can therefore increase computational cost without providing a corresponding improvement in useful information.

Principal Component Analysis (PCA) addresses this problem by transforming the original high-dimensional feature data into a smaller set of new variables called **principal components**. These components are arranged according to the amount of variation or information they represent.

The most significant components can be retained, while less important components can be removed. In this way, the system can represent the original feature set using fewer dimensions while preserving the most important information contained in the original data.

For the case study, this provides several benefits:

- **Dimensionality is reduced**, so fewer features need to be processed.
- **Redundancy is reduced**, because correlated information can be represented more compactly.
- **Computational cost decreases**, as subsequent operations work with fewer dimensions.
- **Storage requirements can be reduced** because the feature representation becomes smaller.
- The simplified feature representation can make subsequent image-processing operations more efficient.

However, PCA should be applied carefully because removing too many components can also result in the loss of useful information. Therefore, an appropriate number of principal components should be retained to achieve a balance between dimensionality reduction and information preservation.

Hence, PCA is an appropriate solution for this case because it converts a large and potentially redundant feature set into a **smaller, more informative representation**, thereby improving computational efficiency while retaining the important characteristics of the extracted features.